## Lab 7: Decision Tree Classification

# NAME=ARNAV NARULA

## REG NO: 2547115

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

## Exploring the Iris Dataset

In [10]:
```
iris = load_iris()

X = iris.data
y = iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df["Target"] = y

print("Samples:", X.shape[0])
print("Features:", X.shape[1])

print("\nFeature Names:")
print(iris.feature_names)

print("\nTarget Classes:")
print(iris.target_names)

print("\nFirst Five Records")
print(df.head())

print("\nClass Distribution")
print(df["Target"].value_counts())
```

```
Samples: 150
Features: 4

Feature Names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Target Classes:
['setosa' 'versicolor' 'virginica']

First Five Records
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Target
0       0
1       0
2       0
3       0
4       0

Class Distribution
Target
0    50
1    50
2    50
Name: count, dtype: int64
```

## Train-Test Split

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])
```

```
Training Samples: 120
Testing Samples: 30
```

## Decision Tree Classification

In [4]:
```
model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy")
```

```
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

```
Accuracy
1.0

Confusion Matrix
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

The Decision Tree classifier was trained using the training dataset and used to predict the class labels of the testing dataset. The model was evaluated using Accuracy Score, Confusion Matrix, and Classification Report. These evaluation metrics help measure the classification performance of the model and its ability to correctly classify the Iris flower species.

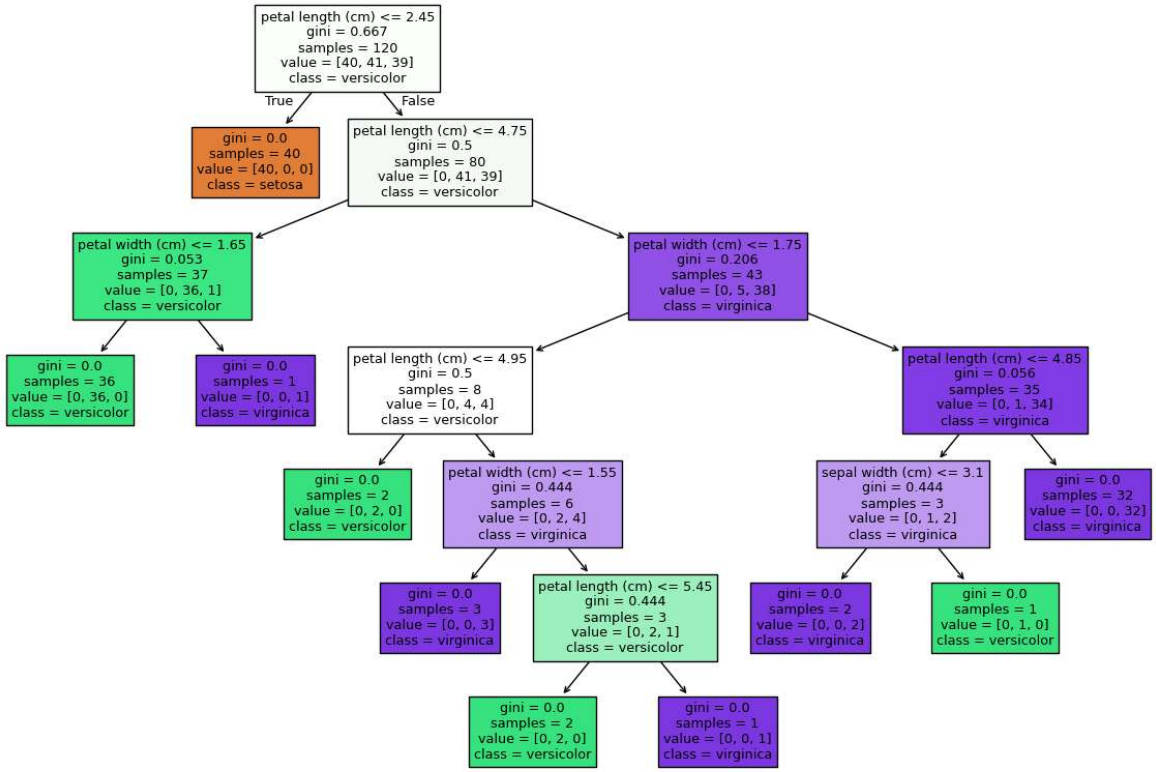
## Decision Tree Visualization

In [5]:
```
plt.figure(figsize=(15,10))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.show()

print("Tree Depth:", model.get_depth())
print("Leaf Nodes:", model.get_n_leaves())
```




```
Tree Depth: 6
Leaf Nodes: 10
```

The trained Decision Tree was visualized using plot_tree(). The tree shows how the model classifies the Iris flowers based on the input features. The maximum tree depth and the number of leaf nodes were displayed to understand the complexity of the model.

## Comparison of Gini Index and Entropy Criteria

In [6]:
```
criteria = ["gini", "entropy"]

results = []

for c in criteria:

    clf = DecisionTreeClassifier(
        criterion=c,
        random_state=42
    )

    clf.fit(X_train, y_train)

    pred = clf.predict(X_test)

    results.append([
        c,
        accuracy_score(y_test, pred),
        clf.get_depth(),
        clf.get_n_leaves(),
        iris.feature_names[clf.tree_.feature[0]]
```

```
    ])

result_df = pd.DataFrame(
    results,
    columns=[
        "Criterion",
        "Accuracy",
        "Tree Depth",
        "Leaf Nodes",
        "Root Feature"
    ]
)

print(result_df)
```

```
  Criterion  Accuracy  Tree Depth  Leaf Nodes       Root Feature
0      gini       1.0           6          10  petal length (cm)
1   entropy       1.0           6          10  petal length (cm)
```

Two Decision Tree models were trained using the Gini Index and Entropy criteria. Their accuracy, tree depth, number of leaf nodes, and root feature were compared. The comparison helps evaluate whether the choice of splitting criterion affects the performance and complexity of the Decision Tree.

## Effect of Maximum Tree Depth on Decision Tree Performance

In [7]:

```
depths = [1,2,3,4,None]

results=[]

for d in depths:

    clf = DecisionTreeClassifier(
        max_depth=d,
        random_state=42
    )

    clf.fit(X_train,y_train)

    train_acc = clf.score(X_train,y_train)

    test_acc = clf.score(X_test,y_test)

    results.append([
        d,
        train_acc,
        test_acc,
        clf.get_depth()
    ])

table=pd.DataFrame(results,
columns=[
"Max Depth",
"Training Accuracy",
```

```
"Testing Accuracy",
"Tree Depth"
])

print(table)
```

```
   Max Depth  Training Accuracy  Testing Accuracy  Tree Depth
0        1.0           0.675000          0.633333           1
1        2.0           0.950000          0.966667           2
2        3.0           0.958333          1.000000           3
3        4.0           0.975000          1.000000           4
4        NaN           1.000000          1.000000           6
```

Decision Tree models were trained using different values of max_depth. The training accuracy, testing accuracy, and tree depth were compared to study how limiting the depth affects model performance. This helps identify the depth that provides the best balance between model complexity and prediction accuracy.

## Effect of min_samples_split on Decision Tree Performance

In [8]:
```
values=[2,5,10,20]

results=[]

for i in values:

    clf=DecisionTreeClassifier(
        min_samples_split=i,
        random_state=42
    )

    clf.fit(X_train,y_train)

    pred=clf.predict(X_test)

    results.append([
        i,
        accuracy_score(y_test,pred),
        clf.get_depth(),
        clf.get_n_leaves()
    ])

table=pd.DataFrame(results,
columns=[
"min_samples_split",
"Accuracy",
"Tree Depth",
"Leaf Nodes"
])

print(table)
```

| | min_samples_split | Accuracy | Tree Depth | Leaf Nodes |
|---|---|---|---|---|
| 0 | 2 | 1.0 | 6 | 10 |
| 1 | 5 | 1.0 | 5 | 8 |
| 2 | 10 | 1.0 | 4 | 6 |
| 3 | 20 | 1.0 | 4 | 6 |

Decision Tree models were trained using different values of min_samples_split. The accuracy, tree depth, and number of leaf nodes were compared to study how this parameter affects the complexity and performance of the model. Higher values generally produced simpler trees with fewer splits.

## Evaluating the Effect of min_samples_leaf in a Decision Tree

In [9]:
```
values=[1,2,5,10]

results=[]

for i in values:

    clf=DecisionTreeClassifier(
        min_samples_leaf=i,
        random_state=42
    )

    clf.fit(X_train,y_train)

    pred=clf.predict(X_test)

    results.append([
        i,
        accuracy_score(y_test,pred),
        clf.get_depth(),
        clf.get_n_leaves()
    ])

table=pd.DataFrame(results,
columns=[
"min_samples_leaf",
"Accuracy",
"Tree Depth",
"Leaf Nodes"
])

print(table)
```

| | min_samples_leaf | Accuracy | Tree Depth | Leaf Nodes |
|---|---|---|---|---|
| 0 | 1 | 1.000000 | 6 | 10 |
| 1 | 2 | 1.000000 | 5 | 8 |
| 2 | 5 | 1.000000 | 4 | 6 |
| 3 | 10 | 0.966667 | 4 | 6 |

Decision Tree models were trained using different values of min_samples_leaf. The accuracy, tree depth, and number of leaf nodes were compared to study how this parameter affects

the complexity and performance of the model. Increasing min_samples_leaf generally reduced the tree complexity and helped control overfitting.

# Task 9: Theory Questions

## 1. What is the role of the criterion parameter in a Decision Tree?

Answer:

The criterion parameter decides how the Decision Tree chooses the best feature to split the data. It uses methods like Gini Index or Entropy to select the split that gives the best classification.

## 2. How does max_depth influence underfitting and overfitting?

Answer:

A small max_depth creates a simple tree that may not learn enough patterns (underfitting). A large max_depth creates a complex tree that may memorize the training data (overfitting).

## 3. Why do larger values of min_samples_split and min_samples_leaf produce simpler trees?

Answer:

Larger values allow fewer splits in the tree. This creates fewer branches and leaf nodes, making the Decision Tree simpler and reducing overfitting.

## 4. Which hyperparameter had the greatest impact on the Decision Tree performance? Support your answer with observations from the experiments.

Answer:

The max_depth parameter had the greatest impact on the Decision Tree. Changing its value clearly affected the accuracy and the size of the tree more than the other parameters.

## 5. Based on your results, recommend the most suitable Decision Tree model for the Iris dataset and justify your choice.

Answer:

The Decision Tree with max_depth = 3 is the most suitable because it gives high testing accuracy while keeping the tree simple. It provides a good balance between accuracy and generalization.